Energy & Buildings 327 (2025) 115075

Contents lists available at ScienceDirect

Energy & Buildings

journal homepage: www.elsevier.com/locate/enbuild

SINERGYM – A virtual testbed for building energy optimization with Reinforcement Learning

Alejandro Campoy-Nieves *, Antonio Manjavacas, Javier Jiménez-Raboso, Miguel Molina-Solana, Juan Gómez-Romero

Department of Computer Science and Artificial Intelligence, Universidad de Granada, Granada, 18071, Spain

ARTICLE INFO

Dataset link: https://github.com/ugr-sail/sinergym

Keywords:
Building Energy Optimization
Simulation
HVAC
EnergyPlus
Machine Learning
Reinforcement Learning

ABSTRACT

Simulation has become a crucial tool for Building Energy Optimization (BEO) as it enables the evaluation of different design and control strategies at a low cost. Machine Learning (ML) algorithms can leverage large-scale simulations to learn optimal control from vast amounts of data without supervision, particularly under the Reinforcement Learning (RL) paradigm. Unfortunately, the lack of open and standardized tools has hindered the widespread application of ML and RL to BEO. To address this issue, this paper presents SINERGYM, an open-source Python-based virtual testbed for large-scale building simulation, data collection, continuous control, and experiment monitoring. SINERGYM provides a consistent interface for training and running controllers, predefined benchmarks, experiment visualization and replication support, and comprehensive documentation in a ready-to-use software library. This paper 1) highlights the main features of SINERGYM in comparison to other existing frameworks, 2) describes its basic usage, and 3) demonstrates its applicability for RL-based BEO through several representative examples. By integrating simulation, data, and control, SINERGYM supports the development of intelligent, data-driven applications for more efficient and responsive building operations, aligning with the objectives of digital twin technology.

1. Introduction

Buildings account for about 30% of global energy consumption and are responsible for 17% and 10% of global direct and indirect global CO_2 emissions, respectively, according to the International Energy Agency[1] and the United Nations Environment Program.[2] The primary source of this consumption comes from Heating, Ventilation and Air Conditioning (HVAC) systems, representing more than 50% of their associated energy demand in developed countries [1]. This consumption is potentially affected by the lack of precise control over these systems, whose proper and efficient operation is essential to ensure energy savings [2–4].

Building Energy Optimization (BEO) aims to reduce energy consumption in buildings while maintaining occupant comfort. BEO methods mainly rely on simulated environments to test, compare and improve energy control strategies that can then be implemented in real buildings. By reducing the risks and costs of testing new strategies, simulation serves as a key enabler of digital twin technology, where virtual models support real-time, intelligent building operations. In this context, Machine Learning (ML) can leverage simulators to automatically learn optimal control strategies from massive data, being often able to outperform traditional methods [5].

Conventional energy analysis systems, such as eQuest[3] and Carrier's Building System Optimizer,[4] are useful for simulating energy consumption and configuring variables of HVAC systems during the design phase. However, these tools rely on static modelling, wherein operational parameters and control schedules remain fixed throughout each simulation. While general-purpose simulators like EnergyPlus[5] and Modelica[6]

* Corresponding author.
E-mail addresses: alejandroac79@correo.ugr.es (A. Campoy-Nieves), manjavacas@ugr.es (A. Manjavacas), jajimer@correo.ugr.es (J. Jiménez-Raboso), miguelmolina@ugr.es (M. Molina-Solana), jgomez@decsai.ugr.es (J. Gómez-Romero).

[1] https://www.iea.org/reports/buildings.
[2] https://www.unep.org/resources/report/2021-global-status-report-buildings-and-construction.
[3] https://www.doe2.com/equest/.
[4] https://www.carrier.com/commercial/en/us/software/hvac-system-design/building-system-optimizer.
[5] https://energyplus.net.
[6] https://modelica.org.

https://doi.org/10.1016/j.enbuild.2024.115075
Received 17 June 2024; Received in revised form 11 November 2024; Accepted 17 November 2024
Available online 22 November 2024
0378-7788/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

offer more flexibility for performance analysis and equipment sizing, they are primarily designed for offline assessing and similarly lack the feedback loops required for continuous control. Specifically, they do not enable real-time adjustments of control strategies or assess the effectiveness of such actions as conditions evolve [6]. This highlights the need for tools that integrate dynamic control features with energy simulation, as demonstrated by BOPTEST for Model Predictive Control in conjunction with Modelica [7].

In the last years, Reinforcement Learning (RL) algorithms have emerged as the predominant approach to data-driven BEO. RL is a paradigm within ML based on learning an optimal behaviour policy through trial and error. This policy must be able to establish a proper mapping between states and actions in order to maximize a reward signal [8]. Deep RL (DRL) extends RL by using neural networks to approximate value functions or policies [9]. RL and DRL have shown better performance than control alternatives such as Rule-Based Controllers (RBC) and Model Predictive Control (MPC) in certain scenarios [10–20], although they face several challenges derived from their data inefficiency [21]. Moreover, the application of RL-based BEO generally requires prior training in a simulated environment, which involves several challenges:

1. **The setup of the building simulation software for different use cases** [6]. As mentioned, simulation engines are not adequate for dynamic and continuous control, and particularly, for the observation-action-reward cycle of RL. Furthermore, configuring these simulators may be challenging because they have heterogeneous communication interfaces and data structures.
2. **The execution and management of a large number of simulations** [11]. A large number of simulations are needed to train RL models adapted to different scenarios, building dynamics, and control strategies, requiring frequent restarts and warm-ups. In addition, these simulations must not only be executed efficiently but also monitored, thus requiring a robust back-end engine and flexible interfaces.
3. **The reproducibility of the results** [16]. Many RL approaches to BEO address similar problems under different, partially defined configurations. Moreover, RL algorithms can exhibit stochastic behaviour, which makes them difficult to compare.
4. **The comparison between controllers' performance** [13]. The absence of an established benchmark for RL-based BEO, in contrast to other ML areas —e.g., image classification, translation, etc.— prevents the standardized assessment of control solutions. Furthermore, as previously mentioned, the different parameters associated with each optimization technique require precise definitions and public disclosure to ensure transparent comparison.

In this paper, we present SINERGYM,[7] an open-source virtual testbed for RL-based BEO that addresses and solves the aforementioned issues. SINERGYM offers a scalable, customizable, and robust platform for configuring, running, tracking, and collecting data from massive building simulations, along with specific features for training DRL agents. The software facilitates the inclusion of additional building models, and promotes the fair comparison between algorithms and the resulting control strategies. By leveraging EnergyPlus for simulation, SINERGYM empowers researchers and practitioners to develop, test, and optimize intelligent control strategies that can transition from virtual environments to real-world building operations. Its unique features have been exploited for comparing state-of-the-art DRL algorithms [22,23], developing new RL approaches [24,25], creating ML datasets [26], learning digital twins [27], explaining control strategies [28], applying federated learning [29], and testing controllers before deploying [30,31], among others.

[7] https://github.com/ugr-sail/sinergym.

The rest of the paper is structured as follows: Section 2 reviews existing frameworks for simulation-based BEO and compares their main features. Section 3 presents the rationale behind the design of SINERGYM, Section 4 details its main functionalities, and Section 5 provides some representative usage examples. Finally, Section 6 summarizes the work and discusses the next steps for the tool.

2. Related work

2.1. Interplay of RL algorithms and building simulators

Simulation-based BEO involves the interaction between a controller and a building simulation engine, such as EnergyPlus and Modelica. Specifically, RL agents interact with the simulated building —the *environment*, in RL terminology— by sending control actions to different building subsystems. As a result, the building transitions to a new state, computed by the simulation engine, resulting in a partial observation of the building state and a reward signal, representing the goodness of the transition.

The Gymnasium API[8] [32] (formerly OpenAI Gym [33]) has become the standard interface for step-by-step interaction between RL agents and simulation environments. It defines three fundamental functions for this interaction: `init`, `step`, and `reset`, along with defined data structures for action and observation spaces. Any RL agent can easily interact with an environment that implements these functions, leading to the widespread adoption of standard DRL libraries, such as StableBaselines3 [34] or RLlib [35]. Since the Gymnasium interface aligns with any control loop, it can be directly used by different types of controllers.

Nevertheless, integrating the Gymnasium API with a building simulator is not straightforward [36,37]. One reason is that building simulation engines are not intended for continuous and extensive interaction, but for running batch simulations during long periods. Furthermore, the execution of building simulation software tends to be costly due to the complex equations of thermodynamic physics laws that have to be solved in real time. Consequently, several frameworks aimed at BEO via building simulations have emerged in recent years. We analyze and compare them in the following subsection.

2.2. RL-oriented frameworks for BEO

All the frameworks presented below aim at simplifying the interaction between custom controllers and back-end building simulators, preferably through the Gym/Gymnasium interface, which abstracts lower-level implementation details. The interaction with the building simulation is also continuous, so control signals are sent with user-defined temporal granularity, while the resulting states can be observed after each control action is performed.

Besides SINERGYM, we identify three widely used frameworks for RL-based building energy optimization: RL Testbed for EnergyPlus [38], BOPTEST-Gym [39], and Energym [40]. In this section, we focus our comparison on these tools, excluding other frameworks that appear to be no longer actively developed, such as Gym-Eplus [10], ModelicaGym [41], Tropical Precooling Environment [42], COmprehensive Building Simulator (COBS) [43], and RL-EmsPy.[9] Additionally, we discuss the role of CityLearn [44], which targets building energy coordination and demand response across cities. We exclude tools like GridLearn [45] and Grid2Op [46], as they focus on energy distribution grids rather than building-specific applications.

Table 1 and Table 2 present a summary of the main features of the studied frameworks. They all include a Python layer over the simulation software to abstract the lower-level implementation details and enable continuous interaction. They rely either on Modelica or EnergyPlus as

[8] https://gymnasium.farama.org/index.html.
[9] https://github.com/mechyai/RL-EmsPy.

Table 1
Comparison between SINERGYM and alternative building control frameworks – Part 1.

Framework	Simulator	Middleware	API	Buildings	Envs
RL TestBed	EnergyPlus v9.5.0	Custom patch in simulator	Gym v0.15.7	1	3
Energym	EnergyPlus v9.5.0 and Modelica v2.14	Functional Mock-up Interface (FMI)	No	7	14
BOPTEST-Gym	Modelica v4.0.0	HTTP REST API (based on FMI)	Gymnasium v0.28.1	7	7
SINERGYM	EnergyPlus v24.1.0	EnergyPlus Python API v0.2	Gymnasium v0.29.1	4	87

Simulator: Building simulation engine (EnergyPlus, Modelica or both). **Middleware**: Communication interface between Gym/Gymnasium and the simulator. **API**: Is the tool compliant with the Gym/Gymnasium interface? **Buildings**: Number of different buildings included. **Envs**: Number of predefined environments (buildings & configuration parameters).

Table 2
Comparison between SINERGYM and alternative building control frameworks – Part 2.

Framework	WeatherSet	WeatherVar	Actions	DynamicSp	CustomRw
RL TestBed	No	No	Continuous	No	No
Energym	Partial	No	Continuous	No	Yes
BOPTEST-Gym	No	No	Both	Yes	Yes
SINERGYM	Yes	Yes	Both	Yes	Yes

WeatherSet: Can each building be used with different weather configurations? **WeatherVar**: Does the temperature dataset vary between episodes? **Actions**: Which action spaces are supported? (discrete, continuous or both). **DynamicSp**: Dynamic spaces; can an environment be configured with different action and observation spaces than the predefined? **CustomRw**: Can custom rewards be defined?

back-end simulators, with Energym being the only one including both of them. All but Energym are compatible with the Gym interface. However, only SINERGYM and BOPTEST-Gym are fully compatible with the latest versions of the underlying simulation engines and the Gymnasium interface, which is currently the default standard in the RL domain.

Note also how a *Middleware* software is always required to convert continuous control actions into building actuator signals passed through the Gym/Gymnasium interface. RL Testbed patches the simulator source code, which makes it low scalable, difficult to update, and more difficult to use and adapt. Energym relies on the Functional Mock-up Interface (FMI),[10] which defines an interface to exchange dynamic simulation models aimed at co-simulation, similarly to the Gym interface. The BOPTEST project encompasses two components: a RESTful HTTP API based on the FMI that facilitates the simulation processes, and a Gymnasium-compliant interface to this API (namely, BOPTEST-Gym). In contrast, SINERGYM uses the EnergyPlus Python API,[11] a recent extension of the simulator to access to its functionalities, making this interaction more efficient.

Regarding the supported buildings and configurations (*Envs* column), every framework offers a list of pre-configured environments, each one including a building model and a weather specification, along with other features such as custom action and observation spaces. These models (*Buildings*) define the architecture and equipment of the building, while the weather conditions are the specifications of the external temperatures and other related variables. Regarding these weather conditions (*WeatherSet*), BOPTEST-Gym and RL Testbed do not allow their modification for a given building, whereas Energym and SINERGYM allow the selection of different weather conditions for each building when setting up new environments. On top of this functionality, SINERGYM implements mechanisms for weather variability (*WeatherVar*), which are explained in detail in subsection 4.2.

RL Testbed and Energym only support continuous *Actions*, whereas BOPTEST-Gym and SINERGYM support continuous and discrete control. RL Testbed and Energym hardwire the allowed action values and different control specifications into the building definition files, which hinders the modification of actionable components and signals. In contrast, SINERGYM and BOPTEST-Gym can programmatically modify several elements of the building definition (*DynamicSp*) —specifically, the observation and action spaces— yielding different scenarios on the fly.

A proper reward function is essential in the training process of RL agents, in order to assess the goodness of actions/states while avoiding undesirable side effects [47]. Since building models have different zones, sensors, actuators, etc., and different metrics to quantify successful control, the customization of reward functions is desirable (*CustomRw*). RL Testbed's function reward is embedded into the code, thus hampering reward customization. BOPTEST supports the definition of key performance indicators (KPIs) via a RESTful service, which can be also used in the reward function of BOPTEST-Gym in addition to the predefined ones. Energym and SINERGYM fully support customized rewards based on any observed variable, being directly specified when the environment is created.

CityLearn distinguishes itself from other frameworks due to its unique purpose and design. Like other tools, it addresses energy optimization, is implemented in Python, and supports RL through a Gymnasium interface. However, CityLearn is specifically aimed at benchmarking multi-agent RL controllers within a virtual smart grid composed of distributed energy resources. While the grid can include buildings, CityLearn abstracts their detailed characteristics and relies on a simplified demand response model, unlike the detailed simulation and control offered by other frameworks. Instead of using a physical simulation, CityLearn employs a data-driven predictive model to determine heating and cooling loads, utilizing a long short-term memory (LSTM) neural network [48]. This eliminates the need for interaction with an external simulator, making CityLearn more self-contained. However, this design can make scenario development and reuse more challenging, particularly because CityLearn lacks a long-established community like those of Modelica and EnergyPlus, where extensive libraries, documentation, and support have accumulated over time. To address this, the authors provide a diverse set of preconfigured environments to facilitate use and experimentation.

[10] https://fmi-standard.org/literature/.
[11] https://energyplus.readthedocs.io/en/latest/api.html.

Table 3
SINERGYM design principles and functionalities.

Requirements	Functionalities
Flexibility to work with multiple and customizable environments	4.1. Building Models 4.2. Weather Types 4.3. Environments
Customizable data processing and model evaluation functions	4.4. Rewards 4.5. Wrappers
Experimentation and benchmarking support	4.6. Controllers 4.7. Benchmarking
Adoption of high quality standards of software development	See repository

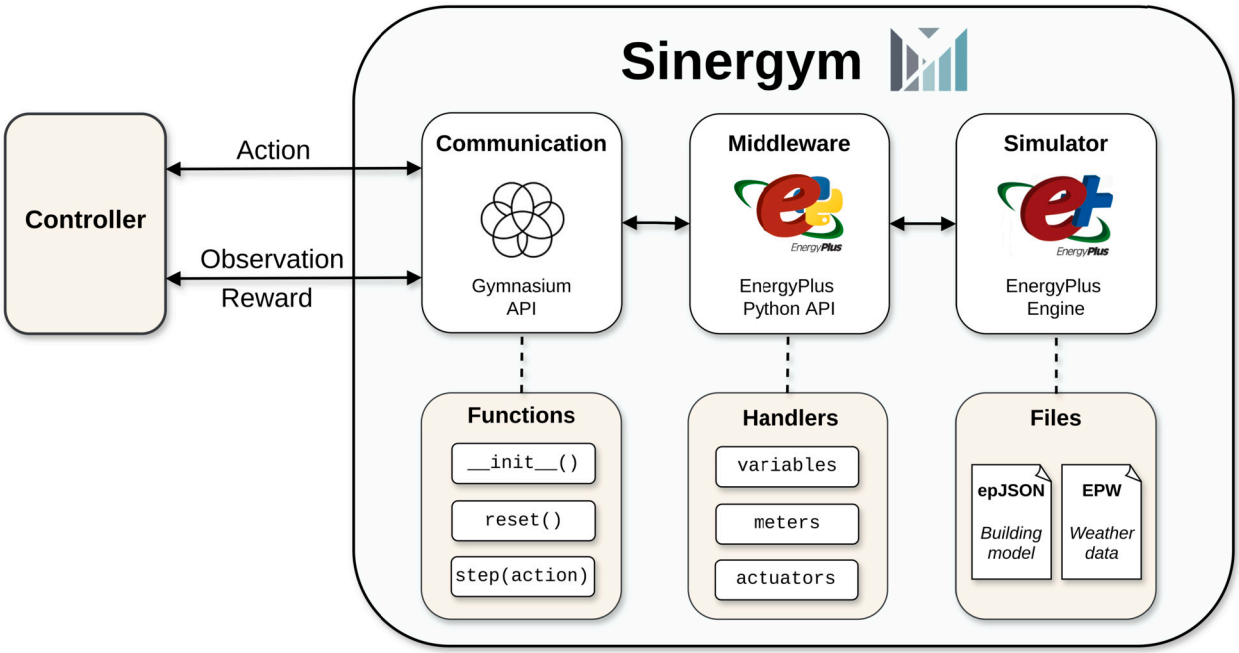


Fig. 1. SINERGYM's general architecture, depicting its three main layers: *communication*, *middleware* and *simulator*.

To summarize, SINERGYM stands out for its ability to integrate new simulated environments and extensively customize the simulation settings. BOPTEST-Gym demonstrates reliability and compatibility with Gymnasium, but it requires a deeper dive into lower-level abstractions for environments modifications, particularly regarding the observation and action spaces. Conversely, RL Testbed is more static, primarily designed for predefined testing scenarios. Finally, while Energym includes interesting features, lacks full compatibility with the Gym/Gymnasium API, thereby limiting its integration capabilities with external optimization and control libraries.

3. Design

3.1. Objectives

SINERGYM is designed to provide broad support for RL control, without excluding alternative control methods. The main objectives behind the design of the tool are the following:

- **Flexibility to work with multiple and modifiable scenarios**. The software must support different buildings and weather configurations, allowing the selection and configuration of controllable elements of the building. Predefined scenarios should be provided for testing and benchmarking purposes.
- **Customizable data processing and model evaluation functions**. The tool must provide means to filter and read building variables to calculate key performance indicators. Furthermore, parametrized reward functions must be employed, together with structures for control actions and state representations allowing simple adaptation to different scenarios.
- **Large-scale experimentation and benchmarking support**. The tool must facilitate the execution of varied environments and guarantee the reproducibility of the results. Experiment parameters, metrics, and outputs must be tracked during the execution and recorded in an usable format for posterior analysis and visualization.
- **Adoption of high-quality standards of software development**. Scientific software sometimes lacks the ease of use and comprehensive documentation of professional tools, which is a barrier for both users and contributing developers. The software must be well-documented to engage a larger community, including illustrative examples that can be executed out of the box and adapted to specific use cases. Installation must be straightforward and based on modern dependency management tools.

We summarize these requirements in Table 3, along with the corresponding functionalities of SINERGYM that will be explained throughout section 4.

3.2. Software architecture

The design of SINERGYM is organized in three layers, as depicted in Fig. 1: *communication*, *middleware*, and *simulator*. The controller

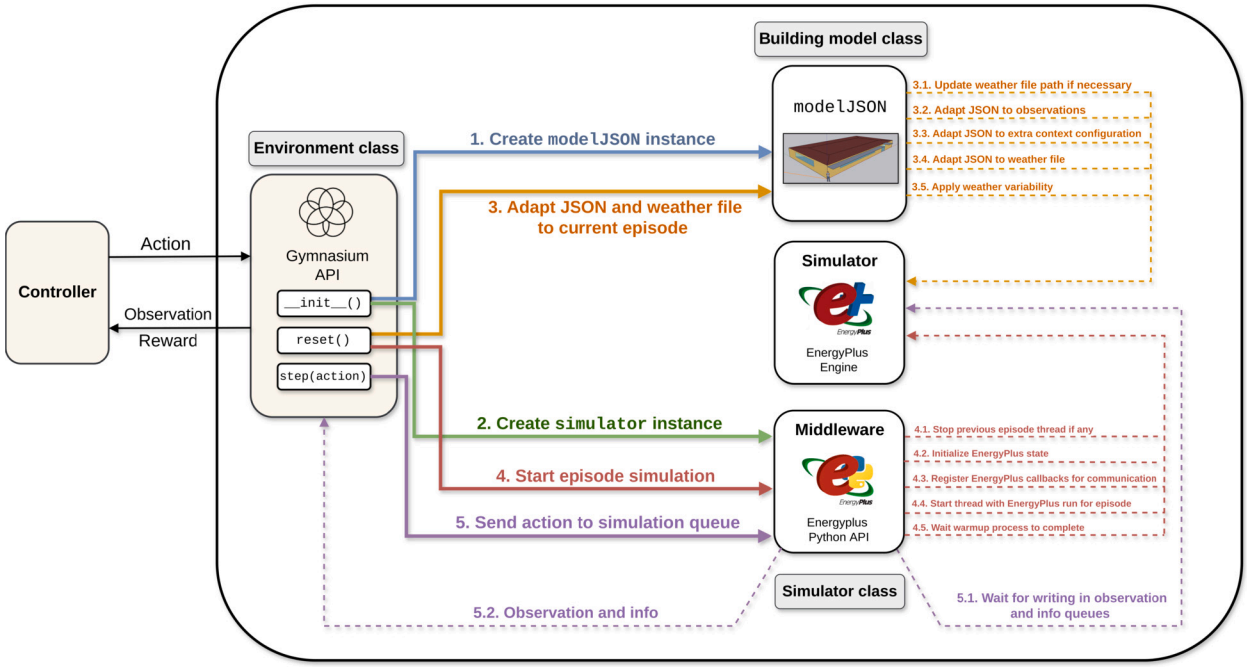


Fig. 2. Overview of the SINERGYM workflow.

sends actions to and receives observations from the simulated building through the Gymnasium interface (*communication* layer). The Gymnasium interface communicates with the simulation engine through the EnergyPlus Python API (*middleware* layer). The EnergyPlus engine runs the simulation (*simulator* layer) and updates the building state. The agent may use the information received from the environment to determine the subsequent control action and to modify its behaviour to maximize future rewards.

Every SINERGYM simulation requires two data files to configure and launch the control process:

- An **epJSON** (*EnergyPlus JavaScript Object Notation*) file, which defines the building model simulated by EnergyPlus, as well as the controllable equipment components. Please refer to subsection 4.1 for further details.
- An **EPW** (*EnergyPlus Weather*) file including the Typical Meteorological Year (TMY3) data to be used in the simulation. More details are provided in subsection 4.2.

SINERGYM encapsulates the components implementing these three layers in an `Environment` class. This class implements the methods of the Gymnasium API, namely `init`, `reset` and `step`. Additional information about the interaction with `Environment` objects is provided in subsection 4.3.

Fig. 2 depicts in detail the internal process flow of the `Environment` class of SINERGYM. In step 1, objects are initialized with `__init__` using the epJSON file passed as a parameter. Next, in step 2, the simulator is launched through the EnergyPlus Python API. This step also involves the creation of handlers for the simulator, which are used to manage and communicate with the underlying engine. Now, the simulator is ready to be launched and the first episode can start after calling `reset`. Before starting each episode, SINERGYM automatically configures some parameters of the simulation (step 3), e.g., noise application on the weather file, definition of building variables for sensors and actuators, or time configuration (steps per hour, run period, etc.).

Step 4 comprises the stages of the simulation cycle: stopping the previous run, if any; setting up the initial state; warm-up process; and the simulation along the run period. While the simulation is running, a controller can interact with the building (step 5). SINERGYM overrides the default building control by overwriting the default EnergyPlus schedulers. When actions are sent to the simulation by calling `step`, SINERGYM transparently interrupts the simulation, sets the control signal, and resumes the process.

4. Functionalities

This section provides an overview of the key functionalities of SINERGYM 3.6.2. For updated information and examples, please refer to its official documentation website.[12]

4.1. Building models

Building models listed in Table 4 are included by default in SINERGYM and are ready to use. Additional experimental buildings and comprehensive information can be found in the project documentation website. The underlying epJSON files are reference models provided by the American Society of Heating, Refrigerating and Air-Conditioning Engineers (ASHRAE)[13] or supported by the U.S. Department of Energy (DOE).[14]

From these buildings, SINERGYM allows the creation of variants on the fly (see subsection 4.3), thus avoiding the manual duplication and edition of epJSON files. These variants include changes to a subset of the model components, which are programmatically defined when the environment is created and internally materialized. Furthermore, SINERGYM offers additional functionalities to facilitate the integration of new building models, such as the automatic detection of sensors and actuators in epJSON files, and the subsequent preparation of these files to be accessed through the Gymnasium interface.

4.2. Weather types

In EnergyPlus, weather data is decoupled from the building models and are read from independent EPW files. SINERGYM includes different weather files provided by EnergyPlus, covering varied climate regions

[12] https://ugr-sail.github.io/sinergym.
[13] https://www.ashrae.org/.
[14] https://www.energycodes.gov/prototype-building-models.

Table 4
Description of the base building models included with SINERGYM.

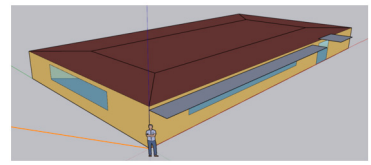
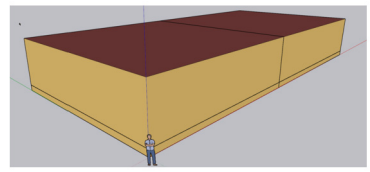
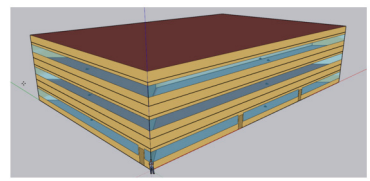
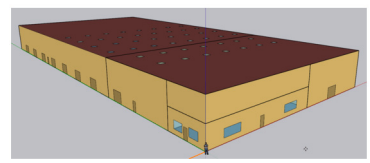
Building	Appearance
5ZoneAutoDXVAV [49,10]: A single-story building divided into 1 indoor and 4 outdoor zones. There are windows on all 4 facades. Its floor area is 463.6 m^2, and it is equipped with a packaged Variable Air Volume (VAV) system, Direct Expansion (DX) cooling coil, and gas heating coils, with fully auto-sized input.	
2ZoneDataCenterHVAC_wEconomizer [50,38]: A 491.3 m^2 building divided into two asymmetrical zones (west and east zone). Each zone has an HVAC system consisting of air economizers, evaporative coolers, DX cooling coil, chilled water coil, and VAV units. The main heat source comes from the hosted servers. The heating and cooling setpoint values are applied simultaneously to both zones.	
ASHRAE9012016_OfficeMedium [48,51]: A 4,979.6 m2 building with three floors and equipped with an HVAC system. Each floor has four perimetral zones and one core zone. Available fuel types are gas and electricity.	
ASHRAE9012016_Warehouse [51]: A non-residential single-storey building with 4,598 m^2, equipped with an HVAC system. The building is divided into three zones: bulk storage, fine storage, and office. The office area is enclosed by the fine storage zone, and it is the only zone with windows. The available fuel types are gas and electricity.	

Table 5
Weather types available in SINERGYM. MAT: mean air temperature; MAH: mean air humidity.

Weather file	Location	Climate type	MAT (°C)	MAH (%)
AUS_NSW.Sydney	Sydney, Australia	Humid subtropical (no dry seasons and hot summers).	17.9	68.83
COL_Bogota	Bogota, Colombia	Mediterranean (dry, warm summers and mild winters).	13.2	80.3
ESP_Granada	Granada, Spain	Mid-latitude dry semiarid and hot dry periods in summer, but passive cooling is possible.	14.84	59.83
FIN_Helsinki	Helsinki, Finland	Moist continental (warm summers, cold winters, no dry seasons).	5.1	79.25
JPN_Tokyo.Hyakuri	Tokyo, Japan	Humid subtropical (mild with no dry season, hot summer).	8.9	78.6
MDG_Antananarivo	Antananarivo, Madagascar	Mediterranean climate (dry warm summer, mild winter).	18.35	75.91
PRT_Lisboa	Lisboa, Portugal	Dry Summer Subtropical Mediterranean (Warm - Marine).	16.3	74.2
USA_AZ_Davis-Monthan	Arizona, USA	Subtropical hot desert (hot and dry).	21.7	34.9
USA_CO_Aurora-Buckley	Colorado, USA	Mid-latitude dry semiarid (cool and dry).	9.95	55.25
USA_IL_Chicago-OHare	Illinois, USA	Humid continental (mixed and humid).	9.92	70.3
USA_NY_New.York-J.F.Kennedy	New York, USA	Humid continental (mixed and humid).	12.6	68.5
USA_PA_Pittsburgh	Pennsylvania, USA	Humid continental (cool and humid).	10.5	66.41
USA_WA_Port.Angeles-William.R.Fairchild	Washington, USA	Cool Marine west coastal (warm summer, mild winter, rain all year).	9.3	81.1

(see Table 5). It also includes functionalities to modify weather data during simulation time, and to integrate custom weather files.

As shown in Fig. 3, SINERGYM implements weather variations by adding noise to the recorded TMY3 temperature. This noise is applied by using an Ornstein-Uhlenbeck (OU) process [52], which is similar to a random walk in continuous time. Following Equation (1), the temperature at the next timestep T_{t+1} is calculated from the previous T_t, with parameters σ (standard deviation of the signal), μ (mean of the signal), τ (time constant), and $W(t_{i+1}) - W(t_i) \sim \mathcal{N}(0, t_{i+1} - t_i)$ (Gaussian noise).

$$T_{t+1} = (1-\mu)T_t - \tau + \sigma(W_{t+1} - W_t) \tag{1}$$

4.3. Scenarios and environments

SINERGYM provides 87 scenarios, each one corresponding to a combination of an epJSON file, an EPW file, selected action and observation spaces, and a predefined reward function. The scenarios can be loaded as environments by specifying the parameter values listed in Table 6. The software automatically creates all the necessary supporting files and data structures, such as the adapted epJSON building file, the weather file with perturbations (if enabled), and the available handlers for the actionable elements of the building.

Two key elements of the environment are: the observation space, which maps variables to the building sensors, and the action space, which maps variables to the building actuators. In particular, the observation space incorporates meters, dates and other output variables from EnergyPlus, while the action space aligns with the EnergyPlus input actuators. Notice that when an actuator is not defined when the `Environment` object is created, SINERGYM uses the default schedulers defined in the base epJSON file of the building model.

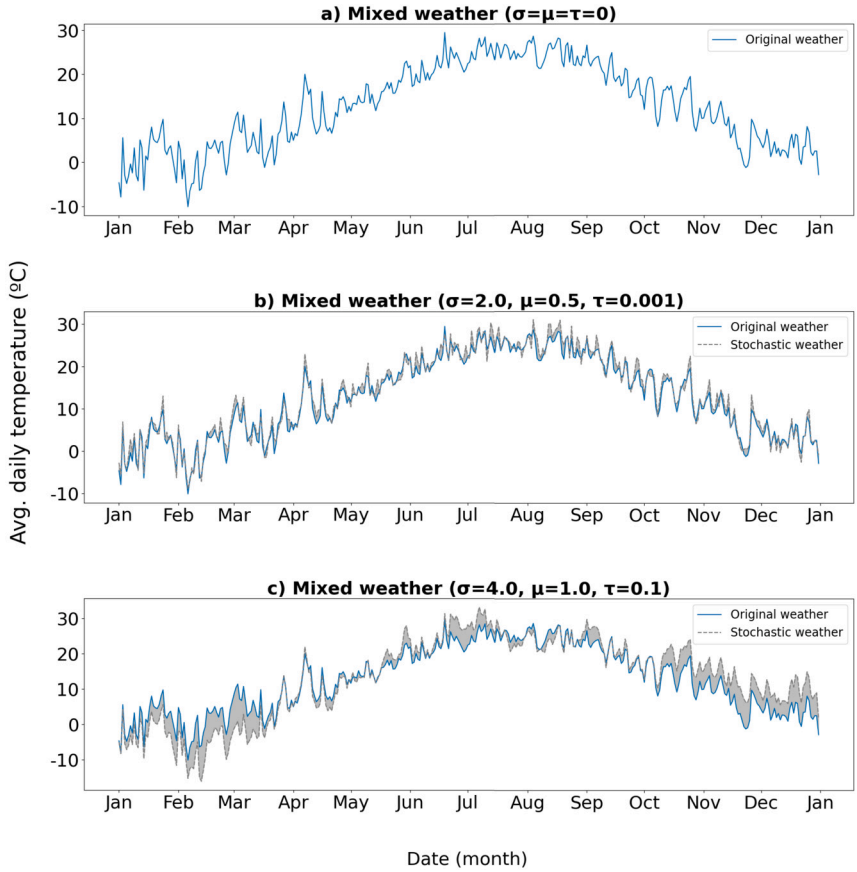


Fig. 3. Ornstein-Uhlenbeck noise applied to a mixed weather dataset (New York) with different values for σ, μ and τ.

4.4. Reward functions

SINERGYM allows users to select between several predefined reward functions and to implement their own custom rewards. If not specified, the default reward function linearly combines energy consumption and thermal discomfort with user-defined weights and comfort ranges, as represented by Equation (2):

$$r_t = -\omega\ \lambda_P\ P_t - (1-\omega)\ \lambda_T\ (|T_t - T_{up}| + |T_t - T_{low}|) \tag{2}$$

where P_t represents power consumption (W); T_t is the current indoor temperature (°C); T_{up} and T_{low} are the imposed comfort range limits (penalty is 0 if T_t is within this range); ω is the weight assigned to power consumption (and consequently, $1-\omega$, the comfort weight), and λ_P and λ_T are scaling constants for consumption and comfort, respectively. Note how the reward function is expressed in negative terms, as we look for its maximization.

SINERGYM includes different alternatives to this default linear reward. For instance, in the *exponential reward*, the discomfort penalty is calculated by an exponential function, thus increasing the cost of out-of-range indoor temperatures. The *schedule reward* uses an additional weighting to modulate the discomfort term, which varies depending on the building occupancy. Furthermore, users can create more sophisticated reward functions involving building-specific variables during the environment definition step.

Finally, as illustrated in Fig. 4, the implementation of the reward function also makes the values of the terms used in its calculation available.

4.5. Wrappers

Gymnasium wrappers are a convenient way to enrich the interaction with an environment without modifying it. More precisely, wrappers are used to process the data exchanged between the agent and the environment without coding. SINERGYM includes many commonly used wrappers, such as observation normalization, action discretization, incremental control, observation stacking and logging. These wrappers can be nested to create complex processing pipelines that favour the training and monitoring of the control agent.

4.6. Controllers

Every controller in SINERGYM is a process that sends actions to the environment through the `step` function. Therefore, it can be a random controller, a reactive rule-based system, an RL agent, etc. By default, SINERGYM adapts to any controller aligned with the Gymnasium interface. This includes controllers defined in the epJSON files, manually-defined controllers, or even complex DRL implementations. Adding custom or learnt controllers —particularly by using libraries compliant with the Gymnasium API— is simple and straightforward.

4.7. Benchmarking

SINERGYM supports logging and enables the comparison between multiple experiments' outputs. These favours reproducible results and promote fair comparisons between controllers. Specifically, environments can be monitored either during the training or evaluation of the controllers, by enabling pre-implemented function callbacks. By using platforms such as Weights&Biases [53], users can define the hyperparameters, loggers, and data storage configuration of the trained models, as well as perform visual analysis through a web-based dashboard. The documentation of SINERGYM provides templates to guide the definition and execution of RL training, and to evaluate experiments with Weights&Biases.

Table 6
Configurable parameters in the creation of a SINERGYM environment.

Parameter	Type	Optional?	Description
building_file	`str`	No	Name of the `epJSON` file with the default building definition.
weather_files	`str` or `List[str]`	No	Name of the `EPW` file with weather data. A list of weather files can be specified from which to sample at each episode.
action_space	`gym.spaces.Box` or `Discrete`	Yes	Gymnasium action space definition. Defaults to an empty action space, which corresponds to the baseline building controller.
time_variables	`List[str]`	Yes	EnergyPlus time variables to be observed. The variable name must match the name of the EnergyPlus Data Transfer API method. Defaults to an empty list.
variables	`Dict`	Yes	Specification for EnergyPlus `Output:Variable`. The key name is user-defined, and the tuple must be the original variable name and the output variable key. Defaults to an empty dictionary.
meters	`Dict`	Yes	Specification for EnergyPlus `Output:Meter`. The key name is user-defined, and the value is the original EnergyPlus Meters name. Defaults to an empty dictionary.
actuators	`Dict`	Yes	Specification for EnergyPlus actuators. The key name is user-defined, and the value is a tuple with actuator type, value type and original actuator name. Defaults to an empty dictionary.
weather_variability	`Tuple[float]`	Yes	Tuple with σ, μ and τ of the Ornstein-Uhlenbeck process to be applied to weather data. Defaults to `None`.
reward	`Reward Class`	Yes	Reward function class. Defaults to `LinearReward`.
reward_kwargs	`Dict`	Yes	Parameters to be passed to the reward function. Varies depending on the reward class. Defaults to an empty dictionary.
max_ep_data_store_num	`int`	Yes	Number of last sub-folders with episodic data saved during simulation. Defaults to 10.
env_name	`str`	Yes	Environment name used for working directory creation. Defaults to `'eplus-env-v1'`.
config_params	`Dict`	Yes	Dictionary with extra configuration for the simulator, e.g., maximum simulation time, or environment sampling frequency. Defaults to `None`.

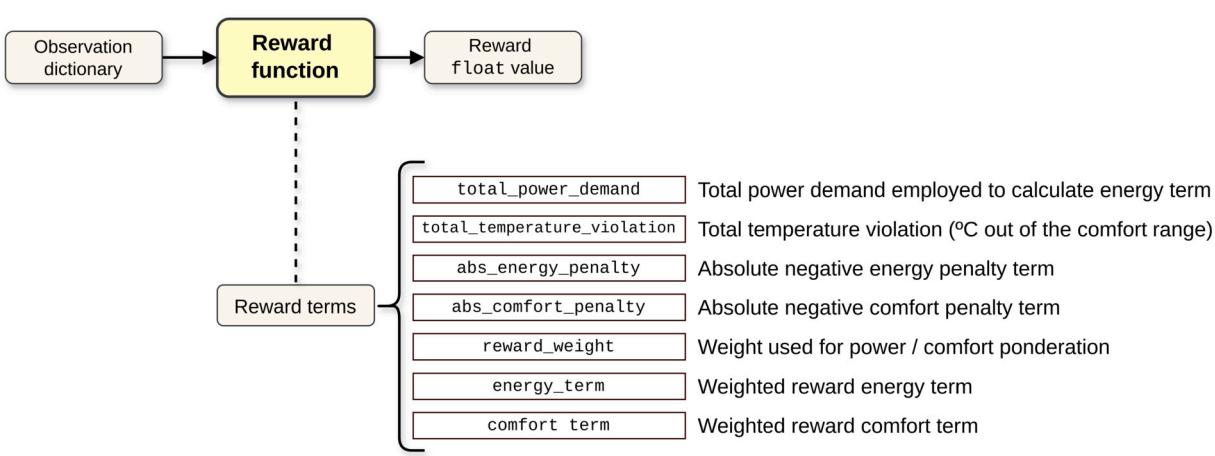


Fig. 4. Example of intermediate values computed by the reward function.

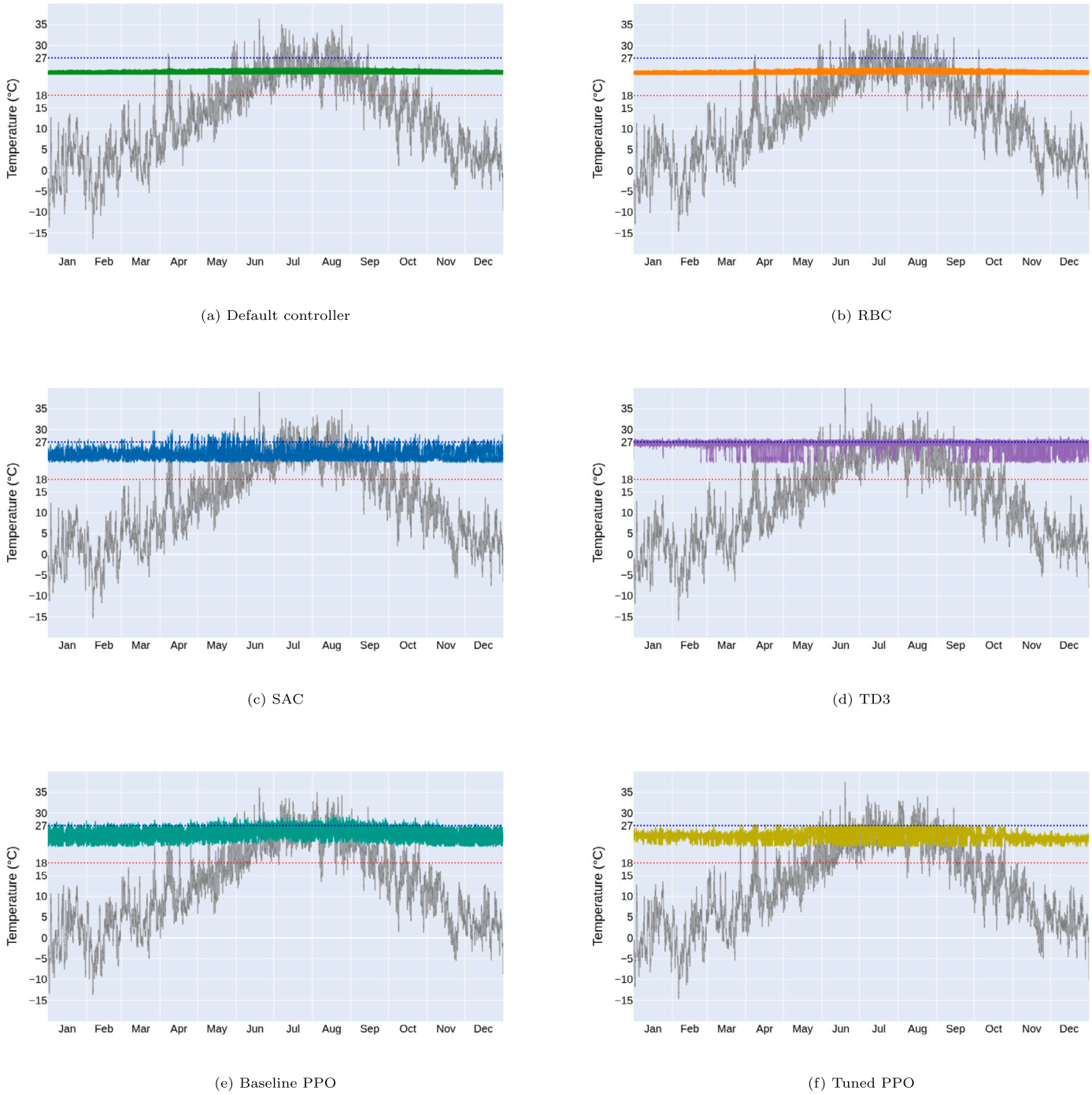
4.8. Installation and running

The installation and configuration of the software are effortless by using PyPi[15] or the Docker distribution.[16] Consequently, SINERGYM can run on a local machine or a cloud computing infrastructure.

[15] https://pypi.org/project/sinergym/.
[16] https://hub.docker.com/r/sailugr/sinergym.

5. Examples

This section showcases the practical application of SINERGYM 3.6.2 through four BEO use cases: 1) testing the default control of a EnergyPlus building model, 2) using a custom rule-based controller, 3) training DRL controllers from scratch, and 4) hyperparameter optimization of the DRL algorithm. The environment is the predefined `Eplus-datacenter-mixed-continuous-stochastic-v1`, featuring the 2ZONEDATA-



(a) Default controller

(b) RBC

(c) SAC

(d) TD3

(e) Baseline PPO

(f) Tuned PPO

Fig. 5. Indoor air temperatures achieved by each controller for a 1-year evaluation period in 2ZoneDataCenterHVAC with mixed weather. The red and blue dotted straight lines indicate the comfort range. Indoor air temperatures are shown in colour, while outdoor temperatures are shown in grey. (For interpretation of the colours in the figure(s), the reader is referred to the web version of this article.)

CenterHVAC building, mixed weather (*USA_NY_New.York-J.F.Kennedy* from Table 5) with noise-added outdoor temperature, and continuous actions. The description of the action and observation spaces, wrappers, reward function, and hyperparameters are included in Appendix A.

Although the main objective of these examples is not to find the best controller for the given control problem, but to show the possibilities of the tool, we show how DRL agents were able to identify effective control strategies without the need for expert knowledge or detailed modelling.

5.1. Example 1: default building controller

In this case, the environment is instantiated with an empty action space, in order to use the default control defined in the building model file. The default control for the 2ZoneDataCenterHVAC temperature setpoints is straightforward, as it statically sets the heating setpoint to 20 °C, and the cooling setpoint to 23 °C for each zone.

Despite its simplicity, the default controller manages to keep the indoor temperature inside the defined comfort range [16, 27]°C dur-

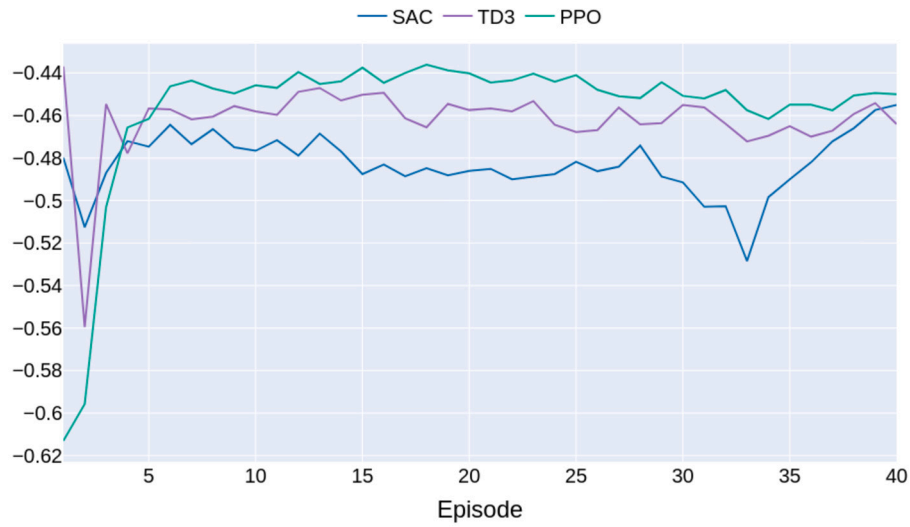


Fig. 6. Mean reward of DRL agents during training.

ing a full-year evaluation period, as depicted in Fig. 5. As expected, since this is a static and fully reactive control, the energy consumption is higher than that of a consumption-aware controller, as discussed in subsection 5.5.

5.2. Example 2: rule-based controller

After testing the default controller, a simple RBC was implemented for the same environment. The input of the RBC is the averaged indoor temperature of the building zones, obtained from the observation space. If this value is above or under the comfort range, the controller respectively increases or decreases the setpoints by one degree (°C). The implementation of this controller is available in the SINERGYM class `RBCIncrementalDatacenter`, included in the `sinergym.utils.controllers` module. This code can be extended to implement other rules, or be adapted to other environments with different observations and actions.

The RBC controller was also able to keep the indoor temperature within the comfort range in the evaluation period, as shown in Fig. 5b, offering a similar performance to the default controller in terms of comfort and energy consumption. Moreover, in contrast to the default controller, RBC does not require a direct modification of the building model, as its implementation is external and independent. This provides a more user-friendly reactive control alternative that is adaptable to complex scenarios.

5.3. Example 3: DRL controllers

We evaluated three DRL algorithms: Soft Actor Critic (SAC) [54], Twin Delayed Deep Deterministic Policy Gradient (TD3) [55], and Proximal Policy Optimization (PPO) [56]. Their implementation was based in the StableBaselines3 library, which can be easily used with SINERGYM. Each agent was trained during 40 episodes, each one corresponding to a 1-year simulation.

Using SINERGYM's feature for intermediate evaluations, we tested the agent's performance every 5 training episodes. This enabled us to monitor the progression of reward and BEO metrics during learning, as well as save the best-performing model obtained throughout the process. As depicted in Fig. 6, the reward generally started stabilizing after episode 7 for all the algorithms. While PPO began as the weakest model, it quickly surpassed the others and consistently delivered better results. Overall, PPO proved to be the most stable learning algorithm in this case, followed by TD3 and SAC.

Fig. 5c, Fig. 5d and Fig. 5e show that the behaviour of the DRL agents notably differed from the default controller and the RBC, allowing temperatures closer to the comfort threshold. The underlying control strategy that they autonomously learned focused on maximizing energy savings while avoiding penalties due to temperature deviations. In this scenario, the penalties for temperature and energy consumption are balanced; adjusting the reward function weights could prioritize one term over the other, generating new policies automatically without expert knowledge. Although the hyperparameters have not been optimized yet, Fig. 9a show that DRL achieved energy savings in the range of 15-18% during several months when compared to the reactive control alternatives. This will be discussed in subsection 5.5.

5.4. Hyperparameter optimization

Finally, we selected the controller trained with PPO, as it proved to be the most stable, as a baseline for further optimization. We initiated a Weights&Biases sweep to explore a grid of 108 hyperparameter combinations (Table A.10), executed in parallel on Google Cloud using the containerized version of SINERGYM. Comprehensive training and output data—including weights, logs, final states, hyperparameters, actions, observations, rewards, and BEO metrics—were recorded in Weights&Biases.[17] The performance of the best PPO configuration is illustrated in Fig. 5f —the detailed results of the top 10 PPO configurations are included in Table B.11. Notably, this process could also be applied to optimize other parameters not directly tied to the algorithm, such as the interval between control actions, the reward function, or the selection of observed or action variables.

5.5. Discussion

The controllers were compared in terms of rewards for 10 episodes after training finished. Fig. 7 represents the rewards obtained by each one. As previously mentioned, PPO achieved the best and most consistent results. The default controller and the RBC showed similar results, with RBC being slightly more stable against different conditions. On the other hand, SAC performed similar to RBC and the default controller, while TD3 generally achieved a better performance than both of them.

Since the reward function combines both the comfort and the energy terms, it is interesting to analyze the values of each one separately. Accordingly, indoor temperatures are presented in Fig. 5 and energy demand is depicted in Fig. 8 and Fig. 9.

The demand values represented in Fig. 8 do not show major differences among the controllers. However, if we express them as per-

[17] https://wandb.ai/sail_ugr/sinergym_paper.

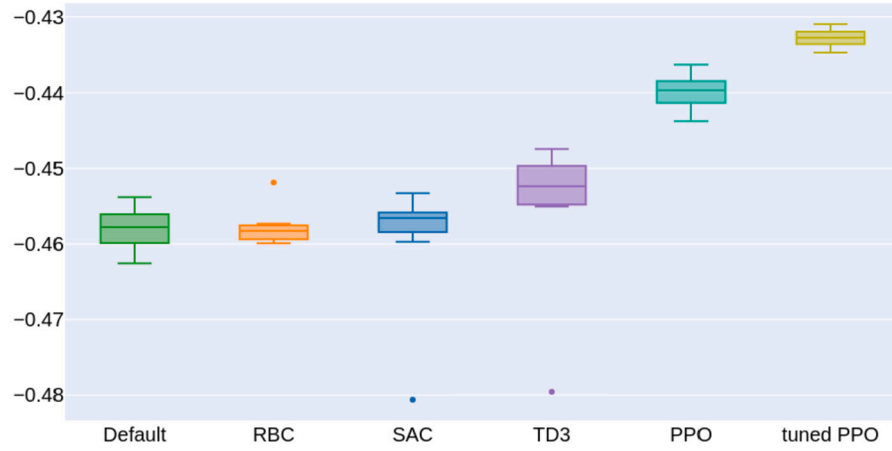


Fig. 7. Mean reward obtained by Default, RBC, SAC, TD3, PPO and optimized PPO controllers during evaluation (10 episodes).

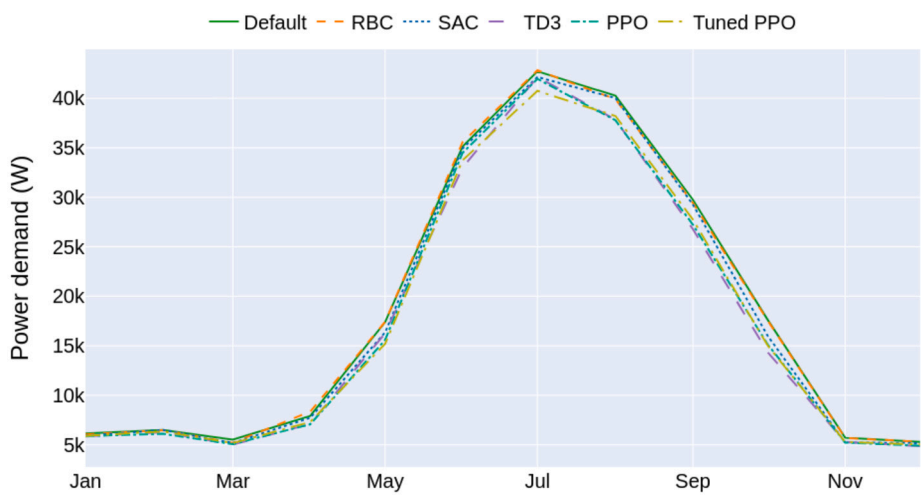


Fig. 8. Monthly average power demand for each controller.

centages, the results in Fig. 9a reveal the energy savings obtained by the DRL agents with respect to the default and the RBC controllers. Savings were mostly achieved during the intermediate seasons, from March to May and from September to November, when the temperatures are mild and the HVAC operation offers more opportunities for optimization. Also, TD3 achieved remarkable energy savings in March, September, October, and December, significantly outperforming PPO. However, PPO generally demonstrated better control over the building's internal temperature, as well as greater stability against induced climate perturbations. The optimized PPO, in turn, obtained significant total savings with respect to the default and the RBC controllers, as shown in Fig. 9b.

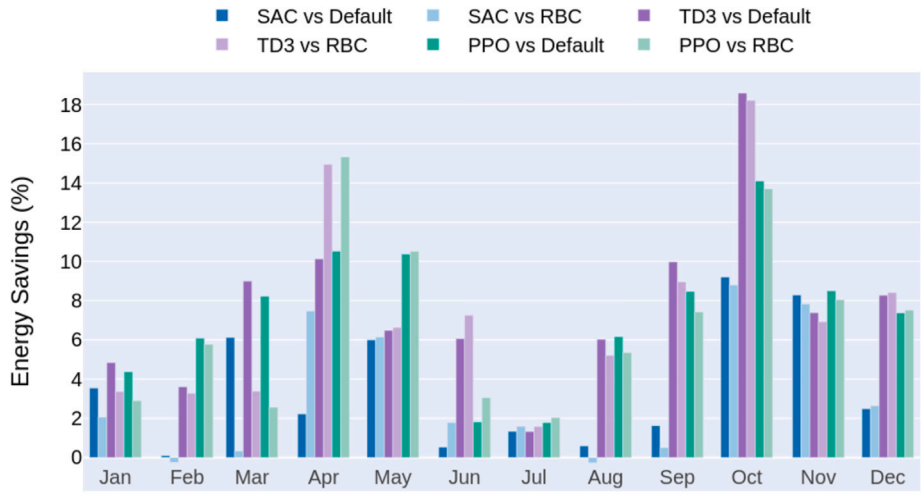
As a side observation, we found that the percentage of time an agent drives indoor temperature outside the comfort zone is not a consistently reliable metric for evaluating controller performance. It is often better to allow temperatures to remain slightly outside the comfort zone for a bit longer, rather than having brief but significant violations. In contrast, the average temperature violation metric provided by SINERGYM offers a more accurate assessment of the agent's deviation from the comfort range and delivers a clearer picture of its overall performance.

6. Conclusions

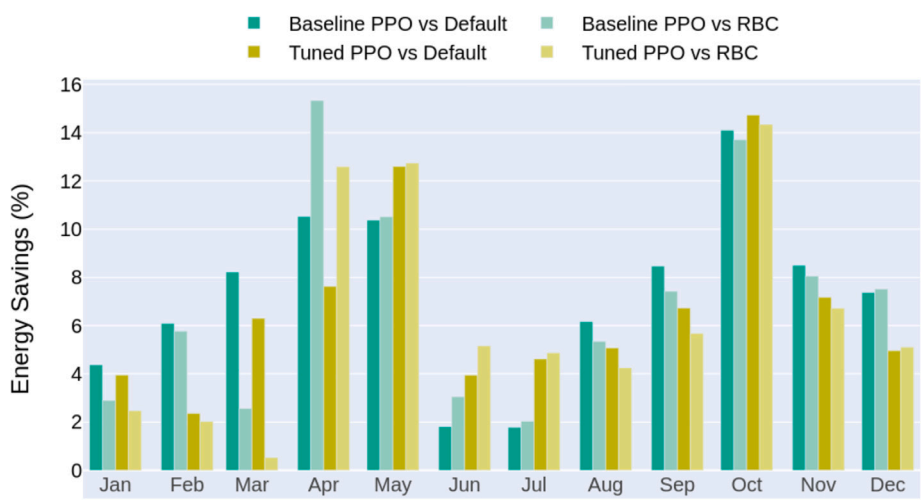
Developing and testing energy control strategies through simulation is becoming the norm in the building operation sector, as it reduces real-world experimentation risks and implementation costs. Extensive simulation is essential in RL, as agents require massive amounts of data to train effectively. As the sector advances toward intelligent, data-driven operations through digital twin technology, the demand for robust simulation platforms continues to grow. While a few existing frameworks leverage building simulation engines to support RL training, most are still in early development or lack the core functionalities necessary for integration into these advanced systems.

In this paper, we have presented SINERGYM, a mature building simulation and control framework for RL-based BEO. It offers a scalable, customizable, and robust platform for configuring, running, and tracking massive building simulations, including numerous valuable features for RL. Specifically, the framework provides custom rewards, controllers, wrappers, continuous and discrete action spaces, support for existing RL libraries, and experiment monitoring and visualization. Moreover, SINERGYM includes several building models that can be adapted and configured by the user, yielding tens of pre-defined environments. SINERGYM represents a significant advancement in the field of twin-enabled building operations and control, offering remarkable ease of use, comprehensive functionalities, robust support, and detailed documentation.

The experiments presented in the paper demonstrate the capabilities of SINERGYM's. In particular, we have showcased the flexibility of the tool in a variety of experimental scenarios, including training and comparison of multiple controllers. The results demonstrate how DRL combined with hyperparameter optimization can lead to robust controllers that do not rely on expert knowledge. This highlights the flexibility and effectiveness of DRL in adapting to complex control problems for BEO,



(a) Energy savings (%): comparison between DRL algorithms versus Default controller and RBC



(b) Energy savings (%): comparison between the baseline PPO and the optimized (tuned) PPO versus Default controller and RBC

Fig. 9. Energy savings achieved by each DRL controller during evaluation. Darker bars represent DRL versus the default controller, while lighter bars represent DRL versus RBC.

as well as SINERGYM's capabilities to automate the configuration and learning processes.

Future work in the short term will be focused on continuing the development of SINERGYM with new features and providing support for the current functionalities. For instance, we will develop a graphical interface to facilitate the configuration and visualization of the experiments. We also plan to add support for additional co-simulation engines, starting from Modelica. The integration of SINERGYM with CityLearn's grid control is also a promising area for future exploration, potentially enhancing coordination between building energy and grid optimization.

Similarly to the widely used Atari and MuJoCo DRL benchmarks, our medium-term goal is to create a consistent benchmark of building environments. This will provide a set of experimental scenarios large enough to support the comprehensive and standardized comparison of different control strategies, thereby supporting future advancements in BEO.

CRediT authorship contribution statement

Alejandro Campoy-Nieves: Writing – original draft, Visualization, Validation, Software, Conceptualization. **Antonio Manjavacas:** Writing – review & editing, Software, Conceptualization. **Javier Jiménez-Raboso:** Writing – review & editing, Software, Conceptualization. **Miguel Molina-Solana:** Writing – review & editing, Supervision, Methodology, Funding acquisition, Conceptualization. **Juan Gómez-Romero:** Writing – review & editing, Supervision, Methodology, Funding acquisition, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Table A.7
Observed variables in 2ZoneDataCenterHVAC.

Variable name	Min.	Max.
Site Outdoor Air Drybulb Temperature	$-5 \cdot 10^6$	$5 \cdot 10^6$
Site Outdoor Air Relative Humidity	$-5 \cdot 10^6$	$5 \cdot 10^6$
Site Wind Speed	$-5 \cdot 10^6$	$5 \cdot 10^6$
Site Wind Direction	$-5 \cdot 10^6$	$5 \cdot 10^6$
Site Diffuse Solar Radiation Rate per Area	$-5 \cdot 10^6$	$5 \cdot 10^6$
Site Direct Solar Radiation Rate per Area	$-5 \cdot 10^6$	$5 \cdot 10^6$
Zone Thermostat Heating Setpoint Temperature	$-5 \cdot 10^6$	$5 \cdot 10^6$
Zone Thermostat Cooling Setpoint Temperature	$-5 \cdot 10^6$	$5 \cdot 10^6$
Zone Air Temperature	$-5 \cdot 10^6$	$5 \cdot 10^6$
Zone Air Relative Humidity	$-5 \cdot 10^6$	$5 \cdot 10^6$
Zone Thermal Comfort Mean Radiant Temperature	$-5 \cdot 10^6$	$5 \cdot 10^6$
Zone Thermal Comfort Clothing Value	$-5 \cdot 10^6$	$5 \cdot 10^6$
Zone Thermal Comfort Fanger Model PPD	$-5 \cdot 10^6$	$5 \cdot 10^6$
Zone People Occupant Count	$-5 \cdot 10^6$	$5 \cdot 10^6$
People Air Temperature	$-5 \cdot 10^6$	$5 \cdot 10^6$
Facility Total HVAC Electricity Demand Rate	$-5 \cdot 10^6$	$5 \cdot 10^6$
Current month	1	12
Current day	1	31
Current hour	0	23

Table A.8
Control actions in 2ZoneDataCenterHVAC.

Variable name	Default scheduler name	Min	Max
Heating Setpoint RL	Heating Setpoints	15.0	22.0
Cooling Setpoint RL	Cooling Setpoints	22.0	30.0

Acknowledgements

This work was funded by ERDF/Junta de Andalucía (D3S project, P21.00247), MICIU/AEI/10.13039/501100011033 (SPEEDY project, TED2021.130454B.I00) and the NextGenerationEU funds (IA4TES project, MIA.2021.M04.0008). A. Manjavacas is funded by ERDF/Junta de Andalucía under SE21_UGR_IFMIF-DONES project.

Appendix A. Settings

This appendix provides details about the configuration of the examples detailed in section 5. In all cases, control actions are performed every 15 minutes —i.e., 4 timesteps per hour— and the episode's run period corresponds to a complete year (from January 1st to December 31st).

Observation space Table A.7 presents the variables returned by the environment as observations.

Action space Table A.8 presents the action space in the 2ZoneDataCenterHVAC building. Both variables are used to control both building zones simultaneously.

Reward function The linear reward function, illustrated in Equation (2), was used to evaluate the performance of the controllers with the parameters defined in Table A.9. The comfort range was defined according to the ASHRAE standard [57], which recommends [18, 27] °C for data centers.

Wrappers The data logging wrapper (`LoggerWrapper`) and the normalization wrappers for the action (`NormalizeAction`) and the observation (`NormalizeObservation`) spaces were applied. The `CSVLogger` and the `WandBLogger` wrappers were enabled to register output data in CSV files and Weights&Biases.

RL algorithm configuration and hyperparameters The hyperparameters and neural network architecture used to train the DRL agents in subsection 5.5 follow the default configurations of StableBaselines3. Specifically, the neural network consists of two fully connected layers with 64 units per layer for PPO, 256 units per layer for SAC, and 400 and 300 units respectively for each layer in TD3.

Table A.10 presents the grid search in the hyperparameter optimization developed with PPO algorithm in subsection 5.4. Each experiment was executed during 20 episodes, with intermediate evaluations every 3 episodes.

Appendix B. Hyperparameter optimization result

Table B.11 shows the performance metrics and the configuration of the top 10 controllers obtained after optimizing PPOs's hyperparameters.

Table A.9
Linear Reward function configuration.

Parameter	Description	Value
`temperature_variables`	List of observed variables which are used to calculate comfort metrics.	West and east zone air temperatures
`energy_variables`	List of the observed variables representing building power demand.	`HVAC_electricity_demand_rate`
`range_comfort_winter`	Temperature comfort range for cold season. Depends on the problem definition.	(18.0, 27.0) °C
`range_comfort_summer`	Temperature comfort range for hot season. Depends on the problem definition.	(18.0, 27.0) °C
`summer_start`	Month and day when summer season starts.	(6, 1)
`summer_final`	Month and day when summer season ends.	(9, 30)
`energy_weight`	Weight for energy reward component. Comfort reward component will be $1 - W_{energy}$	0.5
`lambda_energy`	Constant to calibrate the energy reward component magnitude. Depends on the building.	0.00005
`lambda_temperature`	Constant to calibrate the energy reward component magnitude. Depends on the building.	1.0

Table A.10
Hyperparameter space used for PPO grid search optimization (108 total combinations).

Hyperparameter	Values	Description
learning_rate	0.0001, 0.0003, 0.001	Adjusts the step size for model updates.
gamma	0.9, 0.99, 0.999	Discount factor for future rewards.
batch_size	64, 128	Number of samples per training step.
max_grad_norm	0.5, 0.9	Clips gradients to stabilize training.
ent_coef	0, 0.001, 0.01	Controls the balance between exploration and exploitation.

Table B.11
Top 10 PPO agents based on the best mean reward obtained during on-training evaluations. Values rounded to 4 decimals.

PPO Model	Best reward	Temp. dev. (°C)	Time dev. (%)	Power dem. (W)	Learn. rate	Gam.	Batch size	Max. norm.	Ent. coef.
1st	-0.4310	0.0009	0.9389	17221.1037	0.0003	0.9	128	0.9	0
2nd	-0.4318	0.0059	5.4366	17152.8005	0.0003	0.9	64	0.5	0
3rd	-0.4336	0.0003	0.4937	17338.2957	0.0003	0.9	128	0.9	0.01
4th	-0.4342	0.0017	1.5097	17331.0688	0.0003	0.9	64	0.9	0.01
5th	-0.4346	0.0013	1.4668	17355.1309	0.0003	0.9	64	0.5	0.01
6th	-0.4347	0.0063	5.0884	17262.0097	0.0003	0.9	64	0.9	0
7th	-0.4348	0.0006	0.3881	17380.3400	0.0003	0.9	128	0.9	0.001
8th	-0.4348	0.0027	2.2716	17338.9685	0.0003	0.9	64	0.9	0.001
9th	-0.4351	0.0011	0.6150	17379.9875	0.001	0.9	128	0.5	0.001
10th	-0.4356	0.0035	2.3630	17352.6699	0.0003	0.9	64	0.5	0.001
baseline	-0.4377	0.0043	2.0134	17423.4776	0.0003	0.99	64	0.5	0

Best reward: Best mean reward obtained during on-training evaluations. **Temp. dev. (°C)**: Mean deviation in degrees from comfort range. **Time dev. (%)**: Percentage of simulation time when air temperature deviates from the comfort range. **Power dem. (W)**: Mean Power demand. **Learn. rate**: Learning rate. **Gam.**: Gamma discount factor. **Batch size**: Number of samples. **Max. norm.**: Maximum gradient norm clipping. **Ent. coef.**: Entropy coefficient.

Data availability

Data and software are available at the project repository: https://github.com/ugr-sail/sinergym.

References

[1] L. Pérez-Lombard, J. Ortiz, C. Pout, A review on buildings energy consumption information, Energy Build. 40 (3) (2008) 394–398, https://doi.org/10.1016/j.enbuild.2007.03.007.

[2] C. Wang, K. Pattawi, H. Lee, Energy saving impact of occupancy-driven thermostat for residential buildings, Energy Build. 211 (2020) 109791, https://doi.org/10.1016/j.enbuild.2020.109791.

[3] V.J. Mawson, B.R. Hughes, Optimisation of HVAC control and manufacturing schedules for the reduction of peak energy demand in the manufacturing sector, Energy 227 (2021) 120436, https://doi.org/10.1016/j.energy.2021.120436.

[4] M. Gholamzadehmir, C. Del Pero, S. Buffa, R. Fedrizzi, N. Aste, Adaptive-predictive control strategy for HVAC systems in smart buildings – a review, Sustain. Cities Soc. 63 (2020) 102480, https://doi.org/10.1016/j.scs.2020.102480.

[5] D. Rolnick, P.L. Donti, L.H. Kaack, K. Kochanski, A. Lacoste, K. Sankaran, A.S. Ross, N. Milojevic-Dupont, N. Jaques, A. Waldman-Brown, A.S. Luccioni, T. Maharaj, E.D. Sherwin, S.K. Mukkavilli, K.P. Kording, C.P. Gomes, A.Y. Ng, D. Hassabis, J.C. Platt, F. Creutzig, J. Chayes, Y. Bengio, Tackling climate change with machine learning, ACM Comput. Surv. 55 (2) (Feb. 2022), https://doi.org/10.1145/3485128.

[6] A. Findeis, F. Kazhamiaka, S. Jeen, S. Keshav, Beobench: a toolkit for unified access to building simulations for reinforcement learning, in: Procs. Thirteenth ACM International Conference on Future Energy Systems, e-Energy'22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 374–382.

[7] D. Blum, J. Arroyo, S. Huang, J. Drgoňa, F. Jorissen, H.T. Walnum, Y. Chen, K. Benne, D. Vrabie, M. Wetter, L. Helsen, Building optimization testing framework (BOPTEST) for simulation-based benchmarking of control strategies in buildings, J. Build. Perform. Simul. 14 (5) (2021) 586–610, https://doi.org/10.1080/19401493.2021.1986574.

[8] R. Sutton, A. Barto, Reinforcement learning, in: An Introduction, second edition, in: Adaptive Computation and Machine Learning Series, MIT Press, 2018.

[9] H. Dong, Z. Ding, S. Zhang, H. Yuan, H. Zhang, J. Zhang, Y. Huang, T. Yu, H. Zhang, R. Huang, Deep Reinforcement Learning: Fundamentals, Research, and Applications, Springer Nature, 2020, http://www.deepreinforcementlearningbook.org.

[10] T. Wei, Y. Wang, Q. Zhu, Deep reinforcement learning for building HVAC control, in: Proceedings of the 54th Annual Design Automation Conference 2017, DAC'17, Association for Computing Machinery, New York, NY, USA, 2017, pp. 1–6, https://github.com/zhangzhizza/Gym-Eplus.

[11] K. Mason, S. Grijalva, A review of reinforcement learning for autonomous building energy management, Comput. Electr. Eng. 78 (2019) 300–312, https://doi.org/10.1016/j.compeleceng.2019.07.019.

[12] D. Zhang, X. Han, C. Deng, Review on the research and practice of deep learning and reinforcement learning in smart grids, CSEE J. Power Energy Syst. 4 (3) (2018) 362–370, https://doi.org/10.17775/CSEEJPES.2018.00520.

[13] J.R. Vázquez-Canteli, Z. Nagy, Reinforcement learning for demand response: a review of algorithms and modeling techniques, Appl. Energy 235 (2019) 1072–1089, https://doi.org/10.1016/j.apenergy.2018.11.002.

[14] S. Brandi, M.S. Piscitelli, M. Martellacci, A. Capozzoli, Deep reinforcement learning to optimise indoor temperature control and heating energy consumption in buildings, Energy Build. 224 (2020) 110225, https://doi.org/10.1016/j.enbuild.2020.110225.

[15] D. Azuatalam, W.-L. Lee, F. de Nijs, A. Liebman, Reinforcement learning for whole-building HVAC control and demand response, Energy AI 2 (2020) 100020, https://doi.org/10.1016/j.egyai.2020.100020.

[16] L. Yu, S. Qin, M. Zhang, C. Shen, T. Jiang, X. Guan, A review of deep reinforcement learning for smart building energy management, IEEE Int. Things J. 8 (15) (2021) 12046–12063, https://doi.org/10.1109/JIOT.2021.3078462.

[17] A.T.D. Perera, P. Kamalaruban, Applications of reinforcement learning in energy systems, Renew. Sustain. Energy Rev. 137 (2021) 110618, https://doi.org/10.1016/j.rser.2020.110618.

[18] Q. Fu, Z. Han, J. Chen, Y. Lu, H. Wu, Y. Wang, Applications of reinforcement learning for building energy efficiency control: a review, J. Build. Eng. 50 (2022) 104165, https://doi.org/10.1016/j.jobe.2022.104165.

[19] X. Deng, Y. Zhang, Y. Zhang, H. Qi, Towards optimal HVAC control in non-stationary building environments combining active change detection and deep reinforcement learning, Build. Environ. 211 (2022) 108680, https://doi.org/10.1016/j.buildenv.2021.108680.

[20] M.H.B. Mahbod, C.B. Chng, P.S. Lee, C.K. Chui, Energy saving evaluation of an energy efficient data center using a model-free reinforcement learning approach, Appl. Energy 322 (2022) 119392, https://doi.org/10.1016/j.apenergy.2022.119392.

[21] Z. Nagy, G. Henze, S. Dey, J. Arroyo, L. Helsen, X. Zhang, B. Chen, K. Amasyali, K. Kurte, A. Zamzam, H. Zandi, J. Drgoňa, M. Quintana, S. McCullogh, J.Y. Park, H. Li, T. Hong, S. Brandi, G. Pinto, A. Capozzoli, D. Vrabie, M. Bergés, K. Nweye, T. Marzullo, A. Bernstein, Ten questions concerning reinforcement learning for building energy management, Build. Environ. 241 (2023) 110435, https://doi.org/10.1016/j.buildenv.2023.110435.

[22] M. Wang, J. Willes, T. Jiralerspong, M. Moezzi, A comparison of classical and deep reinforcement learning methods for HVAC control, https://doi.org/10.48550/arXiv.2308.05711, 2023.

[23] A. Manjavacas, A. Campoy Nieves, J. Jiménez-Raboso, M. Molina-Solana, J. Gómez-Romero, An experimental evaluation of deep reinforcement learning algorithms for

HVAC control, Artif. Intell. Rev. 57 (06.2024), https://doi.org/10.1007/s10462-024-10819-x.

[24] A. Dmitrewski, M. Molina-Solana, R. Arcucci, CntrlDA: a building energy management control system with real-time adjustments. Application to indoor temperature, Build. Environ. 215 (2022) 108938, https://doi.org/10.1016/j.buildenv.2022.108938.

[25] Z. An, X. Ding, A. Rathee, W. Du, CLUE: Safe model-based RL HVAC control using epistemic uncertainty estimation, in: Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, BuildSys'23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 149–158.

[26] H.-Y. Liu, X. Fu, B. Balaji, R. Gupta, D. Hong, B2RL: an open-source dataset for building batch reinforcement learning, in: Proceedings of the 9th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, BuildSys'22, Association for Computing Machinery, New York, NY, USA, 2022, pp. 462–465.

[27] A. Naug, A. Guillen, R. Luna Gutiérrez, V. Gundecha, S. Ghorbanpour, L. Dheeraj Kashyap, D. Markovikj, L. Krause, S. Mousavi, A.R. Babu, S. Sarkar, PyDCM: custom data center models with reinforcement learning for sustainability, in: Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, BuildSys'23, ACM, 2023, pp. 232–235.

[28] J. Jiménez-Raboso, A. Manjavacas, A. Campoy-Nieves, M. Molina-Solana, J. Gómez-Romero, Explaining deep reinforcement learning-based methods for control of building HVAC systems, in: L. Longo (Ed.), Explainable Artificial Intelligence, Springer Nature, Switzerland, Cham, 2023, pp. 237–255.

[29] F. Hagström, V. Garg, F. Oliveira, Employing federated learning for training autonomous HVAC systems, arXiv:2405.00389, http://arxiv.org/abs/2405.00389, 2024.

[30] D. Wölfle, S.R. Vossberg, H. Schmeck, TBMEnv - an environment to assess the accuracy of thermal building models under realistic conditions, in: Proceedings of the 10th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, BuildSys'23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 357–360.

[31] A. Jang, Active reinforcement learning for robust building control, Master's thesis, EECS Department, University of California, Berkeley, May 2023, http://www2.eecs.berkeley.edu/Pubs/TechRpts/2023/EECS-2023-101.html.

[32] A. Kwiatkowski, M. Towers, J. Terry, J.U. Balis, G.D. Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J.J. Tai, H. Tan, O.G. Younis, Gymnasium: a standard interface for reinforcement learning environments, arXiv:2407.17032, https://doi.org/10.48550/arXiv.2407.17032, 2024.

[33] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, W. Zaremba, OpenAI Gym, arXiv:1606.01540 [cs], https://doi.org/10.48550/arXiv.1606.01540, Jun. 2016.

[34] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, Stable-baselines3: reliable reinforcement learning implementations, J. Mach. Learn. Res. 22 (268) (2021) 1–8, https://stable-baselines3.readthedocs.io/en/master/.

[35] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, I. Stoica, RLlib: abstractions for distributed reinforcement learning, in: Proceedings of the 35th International Conference on Machine Learning, ICML 2018, in: Proceedings of Machine, vol. 80, Learning Research, PMLR, Stockholm, Sweden, 2018, pp. 3053–3062, http://proceedings.mlr.press/v80/liang18b.html.

[36] Z. Wang, T. Hong, Reinforcement learning for building controls: the opportunities and challenges, Appl. Energy 269 (2020) 115036, https://doi.org/10.1016/j.apenergy.2020.115036.

[37] M. Biemann, F. Scheller, X. Liu, L. Huang, Experimental evaluation of model-free reinforcement learning algorithms for continuous HVAC control, Appl. Energy 298 (2021) 117164, https://doi.org/10.1016/j.apenergy.2021.117164.

[38] T. Moriyama, G. De Magistris, M. Tatsubori, T.-H. Pham, A. Munawar, R. Tachibana, Reinforcement learning testbed for power-consumption optimization, in: L. Li, K. Hasegawa, S. Tanaka (Eds.), Methods and Applications for Modeling and Simulation of Complex Systems, in: Communications in Computer and Information Science, Springer, Singapore, 2018, pp. 45–59, https://github.com/IBM/rl-testbed-for-energyplus.

[39] J. Arroyo, C. Manna, F. Spiessens, L. Helsen, An OpenAI-gym environment for the building optimization testing (BOPTEST) framework, in: Proceedings of the 17th IBPSA Conference, 2021, p. 9.

[40] P. Scharnhorst, B. Schubnel, C. Fernández-Bandera, J. Salom, P. Taddeo, M. Boegli, T. Gorecki, Y. Stauffer, A. Peppas, C. Politi, Energym: a building model library for controller benchmarking, Appl. Sci. 11 (8) (2021) 3518, https://doi.org/10.3390/app11083518, https://github.com/bsl546/energym.

[41] O. Lukianykhin, T. Bogodorova, ModelicaGym: applying reinforcement learning to modelica models, in: Proceedings of the 9th International Workshop on Equation-Based Object-Oriented Modeling Languages and Tools, EOOLT'19, Association for Computing Machinery, New York, NY, USA, 2020, pp. 27–36, https://github.com/ucuapps/modelicagym.

[42] D. Wölfle, A. Vishwanath, H. Schmeck, A guide for the design of benchmark environments for building energy optimization, in: Procs. 7th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, BuildSys'20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 220–229, https://github.com/fzi-forschungszentrum-informatik/tropical_precooling_environment.

[43] T. Zhang, O. Ardakanian, COBS: Comprehensive Building Simulator, in: Proceedings of the 7th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, BuildSys'20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 314–315.

[44] K. Nweye, K. Kaspar, G. Buscemi, T. Fonseca, G. Pinto, D. Ghose, S. Duddukuru, P. Pratapa, H. Li, J. Mohammadi, L.L. Ferreira, T. Hong, M. Ouf, A. Capozzoli, Z. Nagy, CityLearn v2: energy-flexible, resilient, occupant-centric, and carbon-aware management of grid-interactive communities, J. Build. Perform. Simul. (2024) 1–22, https://doi.org/10.1080/19401493.2024.2418813.

[45] A. Pigott, C. Crozier, K. Baker, Z. Nagy, GridLearn: multiagent reinforcement learning for grid-aware building energy management, arXiv:2110.06396 [cs], https://doi.org/10.48550/arXiv.2110.06396, https://github.com/apigott/GridLearn, Oct. 2021.

[46] A. Marot, B. Donnot, G. Dulac-Arnold, A. Kelly, A. O'Sullivan, J. Viebahn, M. Awad, I. Guyon, P. Panciatici, C. Romero, Learning to run a power network challenge: a retrospective analysis, in: Proceedings of the NeurIPS 2020 Competition and Demonstration Track, in: PMLR, ISSN 2640-3498, 2021, pp. 112–132, https://proceedings.mlr.press/v133/marot21a.html.

[47] C. Zhang, O. Vinyals, R. Munos, S. Bengio, A study on overfitting in deep reinforcement learning, https://doi.org/10.48550/ARXIV.1804.06893, 2018.

[48] G. Pinto, D. Deltetto, A. Capozzoli, Data-driven district energy management with surrogate models and deep reinforcement learning, Appl. Energy 304 (2021) 117642, https://doi.org/10.1016/j.apenergy.2021.117642.

[49] X. Ding, W. Du, A.E. Cerpa, MB2C: model-based deep reinforcement learning for multi-zone building control, in: Proceedings of the 7th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation, BuildSys'20, Association for Computing Machinery, New York, NY, USA, 2020, pp. 50–59.

[50] Y. Li, Y. Wen, D. Tao, K. Guan, Transforming cooling optimization for green data center via deep reinforcement learning, IEEE Trans. Cybern. 50 (5) (2020) 2002–2013, https://doi.org/10.1109/TCYB.2019.2927410.

[51] H. Cho, B. Liu, K. Gowri, Energy saving impact of ASHRAE 90.1 Vestibule requirements: Modeling of air infiltration through door openings, Tech. Rep. PNNL-20026, Pacific Northwest National Lab, Richland, WA (United States), 2010.

[52] R. Zárate-Miñano, M. Anghel, F. Milano, Continuous wind speed models based on stochastic differential equations, Appl. Energy 104 (2013) 42–49, https://doi.org/10.1016/j.apenergy.2012.10.064.

[53] L. Biewald, Experiment tracking with Weights and Biases, software available from wandb.com, https://www.wandb.com/, 2020.

[54] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: J.G. Dy, A. Krause (Eds.), Proceedings of the 35th International Conference on Machine Learning, ICML 2018, in: Proceedings of Machine, vol. 80, Learning Research, PMLR, Stockholm, Sweden, 2018, pp. 1856–1865, http://proceedings.mlr.press/v80/haarnoja18b.html.

[55] S. Fujimoto, H. van Hoof, D. Meger, Addressing function approximation error in actor-critic methods, in: J.G. Dy, A. Krause (Eds.), Proceedings of the 35th International Conference on Machine Learning, ICML 2018, in: Proceedings of Machine, vol. 80, Learning Research, PMLR, Stockholm, Sweden, 2018, pp. 1582–1591, http://proceedings.mlr.press/v80/fujimoto18a.html.

[56] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv:1707.06347 [cs], https://doi.org/10.48550/arXiv.1707.06347, Aug. 2017.

[57] ASHRAE, ASHRAE TC9.9: Data center power equipment thermal guidelines and best practices systems, ASHRAE (2016).